Experiment :
Test Smells Refactoring and Detection

Experiment

- **An exercise / experiment involving the learning of some concepts about test quality, including removing test smells by refactoring the test code and learn to detect and recognize test smells in test code**

First Step : Pre-questionnaire

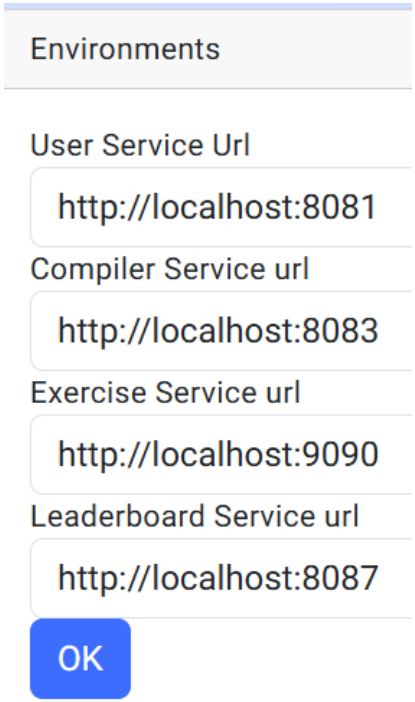
- **All the participants to the course and to the experiment have to fill the following form before working at the experiment (and in any case before December 6°, 2024)**
- **https://forms.office.com/e/X0rnpgDPtg**

Tool configuration

- **The Smell Game tool can be used in several different ways:**
  1. By installing a local Electron client
     - *Useful to have faster responses and to reduce the load on the server*
  2. By directly using it in a browser
  3. By installing the full environment based on Docker Containers on the host machine
  4. This third mode will not be used for this activity except in cases in which all the other modes fail

Tool Configuration

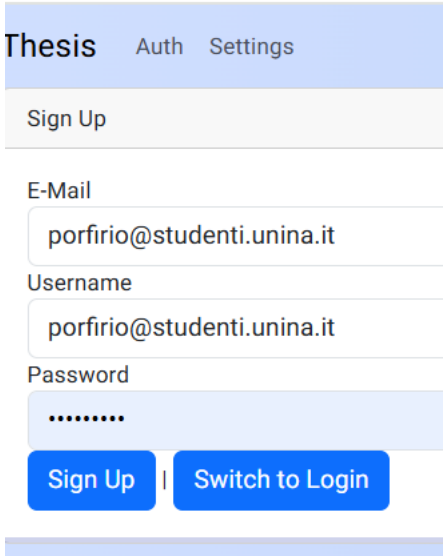
- **All the instructions are included in the Game Instruction document**
- **First of all, in the Settings menu it is needed to set the correct server address for the services**
- **The server addresses will be provided in another file and updated if needed**



Tool configuration

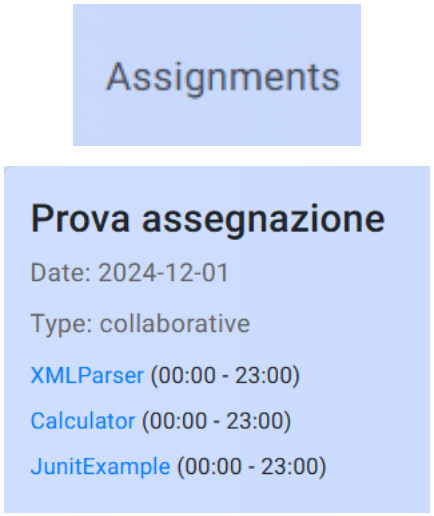
- **Each student has to sign up into the system using his studenti.unina.it e-mail both as username and as email**

- **After the registration, the student has to login and then he has to return to complete Settings**



Second step : Refactoring

- **Each student will receive three test classes including several test smells**
- **The assigned classes can be read by the menu Assignments**
- **Each exercise has start and end time (all the day except last day)**
- **Note: exercises will be «reproposed» each day but obviously a single resolution is requested**



Second step : Refactoring

- **Clicking on an exercise, the main windows of Refactoring Game with the class under test and the test class will be shown**
- **Clicking on Compile the test will be executed and a list of the existing smells will be shown**

- **Note : for some classes some tests could report failures: it is not relevant regarding refactoring**

Refactoring Game GUI

Calculator.java

Fullscreen

```java
package com.dariotintore.tesi;

/**
* Calculator class:
* Basic Mathematical functions like
* Add,Subtract,Multiply,Divide.
*
*/
public class Calculator {
    //no-arg constructor
    public Calculator() {
    }
    /**
    * Sum method.
    */
```

CalculatorTest.java

Fullscreen

```java
package com.dariotintore.tesi;

import org.junit.*;
import static org.junit.Assert.*;



//Arrange-Act-Assert pattern

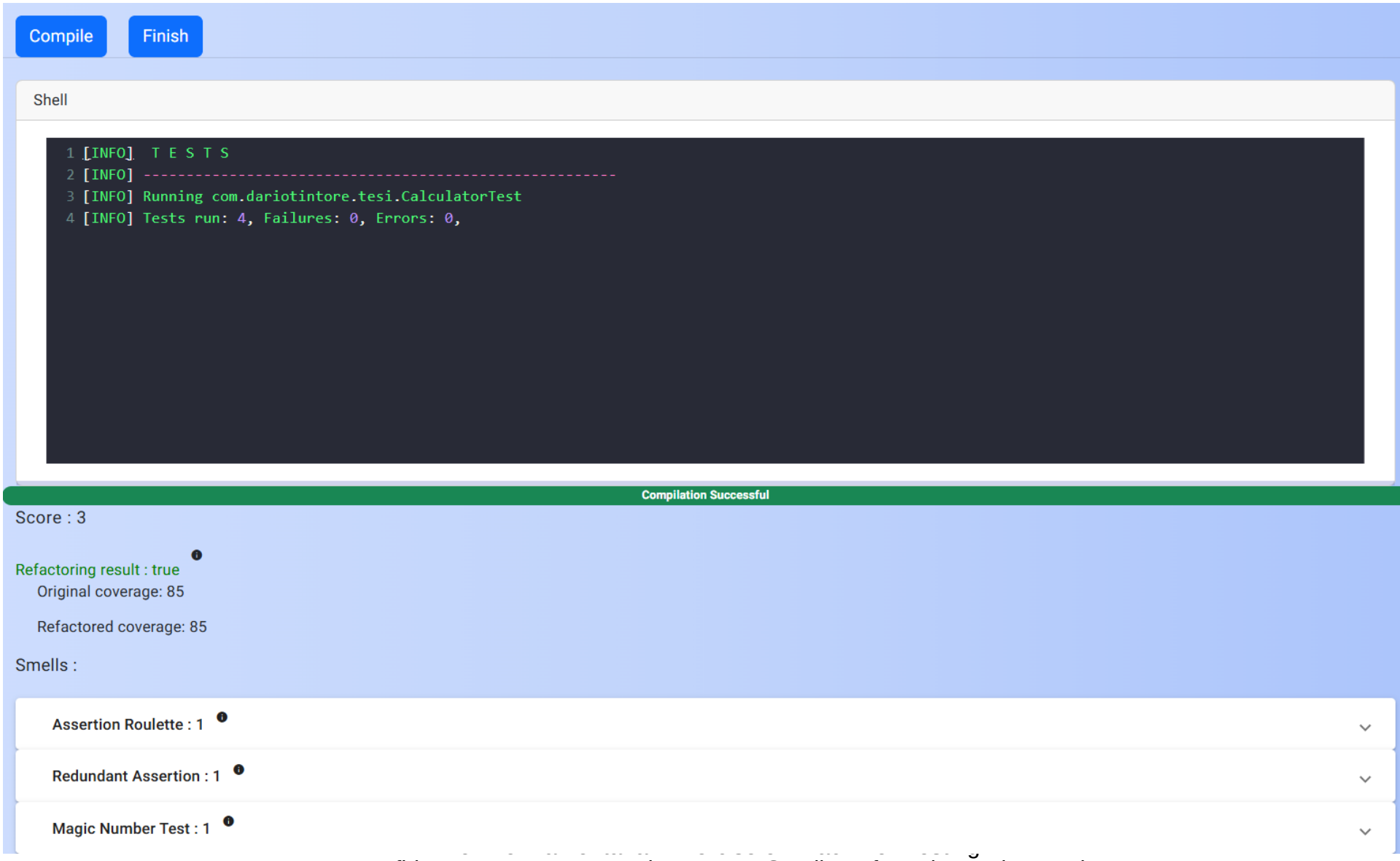
public class CalculatorTest {

private Calculator objCalcUnderTest;

@Before
public void setUp() {
```

Reset

Compile and Find smells



Second step : Refactoring

- **With the help of the tool that notifies in which methods contains test smells, the student has to refactor the test code for removing the smell's cause**
- **After the student has modified the code by clicking Compile it is possible to know if the smell has been removed**

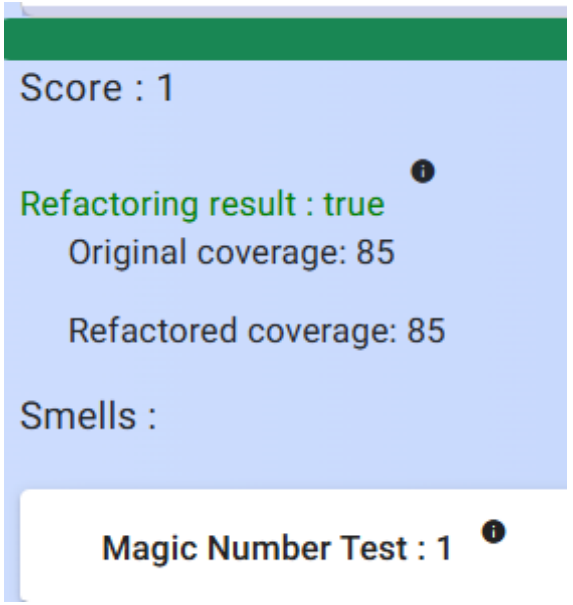
- **Note: smells could be removed simply by deleting code. This solution is considered valid by the system only if there is not a sensible reduction in code coverage**

Example

- **In this example the Redundant Assertion has been refactored by removing it and without losses in code coverage**

CalculatorTest.java

```
19
20      @Test
21      public void testAdd() {
22          int a = 15; int b = 20;
23          int expectedResult = 35;
24          //Act
25          long result = objCalcUnderTest.add(a, b);
26          //Assert
27          assertEquals(expectedResult, result);
28          //assertEquals(result, result);
29      }
30
31      @Test
32      public void testSubtract() {
33          int a = 25; int b = 20;
```



Second step : Refactoring

- **The exercise can be terminated in any moment by clicking Finish**
  - Conventionally, the system considered valid an exercise with few remaining smells (the number of allowed smells is different for each exercise and reported in the activity description)
- **The system will download four files (Class code, Test Code, Shell Code and Results) : please hold them**
  - Please remember the time devoted to each activity
- **The system will return to the main screen, but other solutions to the same exercise can be submitted further: new solutions update the previous solution**

WARNING : problems with assignment

- **In case of problems with assignment feature, it is possible to solve the assigned exercise from the Refactoring game menu**
- **In this case the student MUST autonomously save (by copy and paste in external files) - and send when completing the activity**
  - the refactored test code
  - The number and type of remaining smells

Second step : Refactoring

- **The exercises must be completed before December 9th, 2024**
- **The result files has to be uploaded as answers of the Teams activity**
- **In addition, the following form has to be filled just after the activity submission:**
  - https://forms.office.com/e/hzQ8fATqnW
    - *Some questions are almost duplicated for each experiment: repeat the same answers if needed*

Usefulness questionnaire

- **After sending the results questionnaire, there is a last questionnaire to be compiled**
- **This last questionnaire is about the perceived usefulness of the tool**
  - Its results will be used as feedbacks to evaluate and improve the tool
  - The answers represent your opinions about it and obviously will not be object of evaluation
- **https://forms.office.com/e/Lc5KYuub2L**

Final questionnaire

- **A final exercise about detecting existing smells with a quiz will be carried out in the classroom December 9 th**
- **In the following, instructions for that exercise will be uploaded**